IMSET
INSTITUT MAGHRÉBIN DES SCIENCES ÉCONOMIQUES ET DE TECHNOLOGIE
HONORIS UNITED UNIVERSITIES

AAC
ACADÉMIE D'ART DE CARTHAGE
HONORIS UNITED UNIVERSITIES

Intitulé du cours: Programmation Java

Chapitre 4 :
Héritage
Polymorphisme
Classe abstraite
interface

Filière: 2ème année - BTS-Développement sur Internet

Enseignante: Abid Arij

Année de formation 2024/2025

Plan

Objectifs du module
Introduction
Héritage
Polymorphisme
Classe abstraite
interface

Partie 1
Héritage

Notion de l'héritage

- **Définition de trois classes:**

Personne
nom cin anneeNaiss
age()

Etudiant
nom cin anneeNaiss note1,note2
age() moyenne()

Employe
nom cin anneeNaiss nbHeures prixHeure
age() salaire()

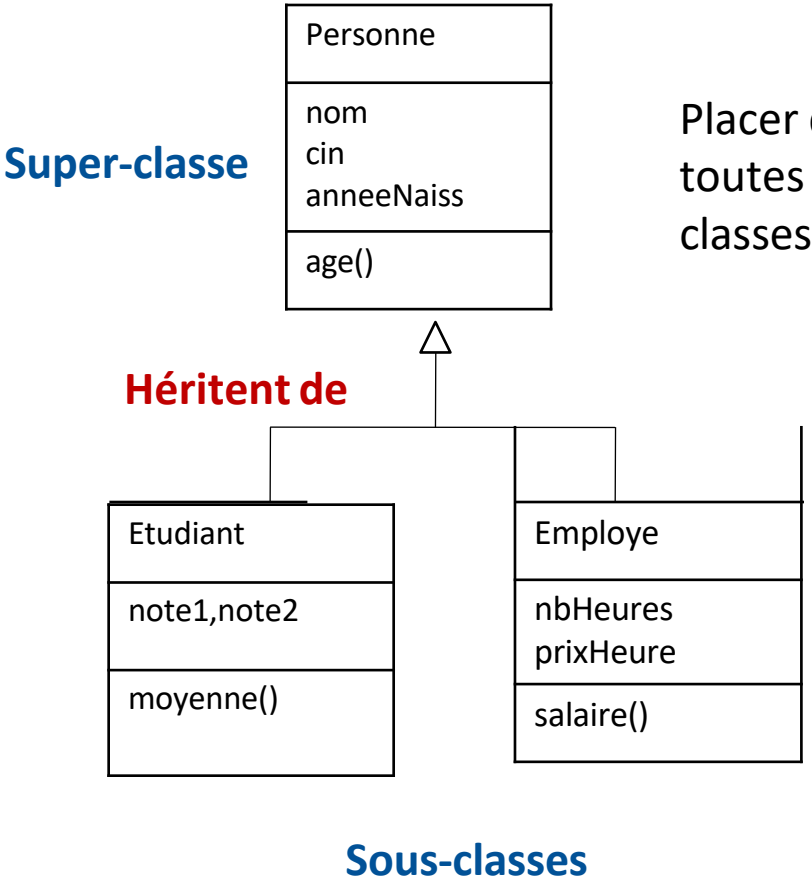
- **Limites:**

Duplication de code.

Une modification faite sur un attribut (ou méthode) commun doit être refaite dans les autres classes.

Notion de l'héritage

Solution



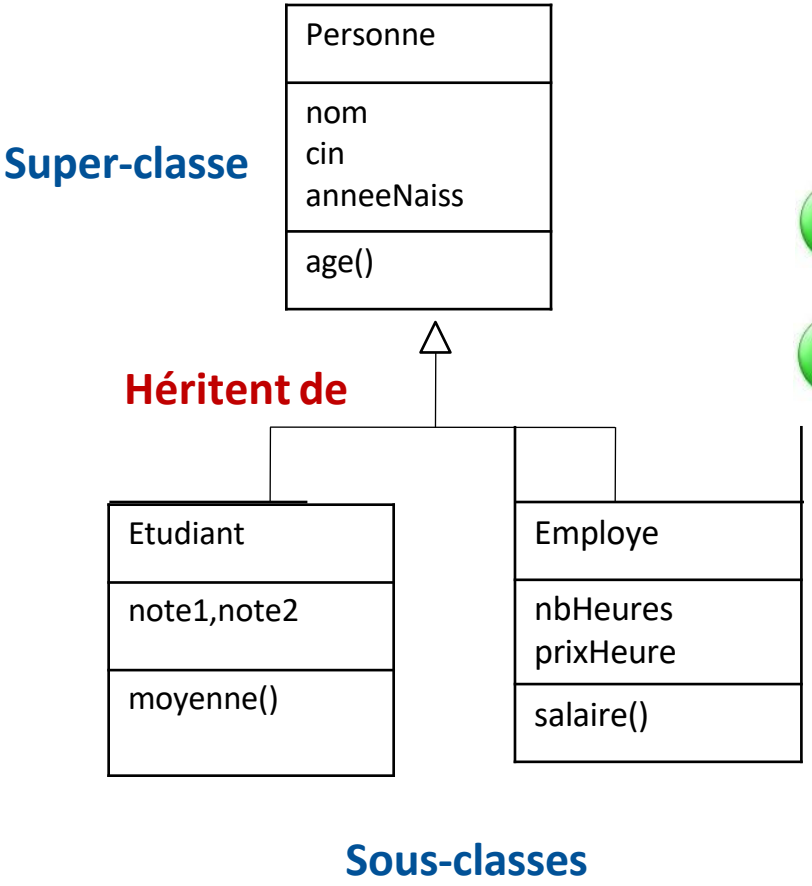
Placer dans la classe mère tous les attributs et toutes les méthodes communs à toutes les classes.

Les classes filles ne comportent que les attributs ou méthodes plus spécifiques.

Les classes filles héritent automatiquement les attributs (et les méthodes) qui n'ont pas besoin d'être ré-écrits.

Notion de l'héritage

Solution



- Réutilisation du code
- Une seule modification des attributs (ou méthodes) en commun

L'héritage en java

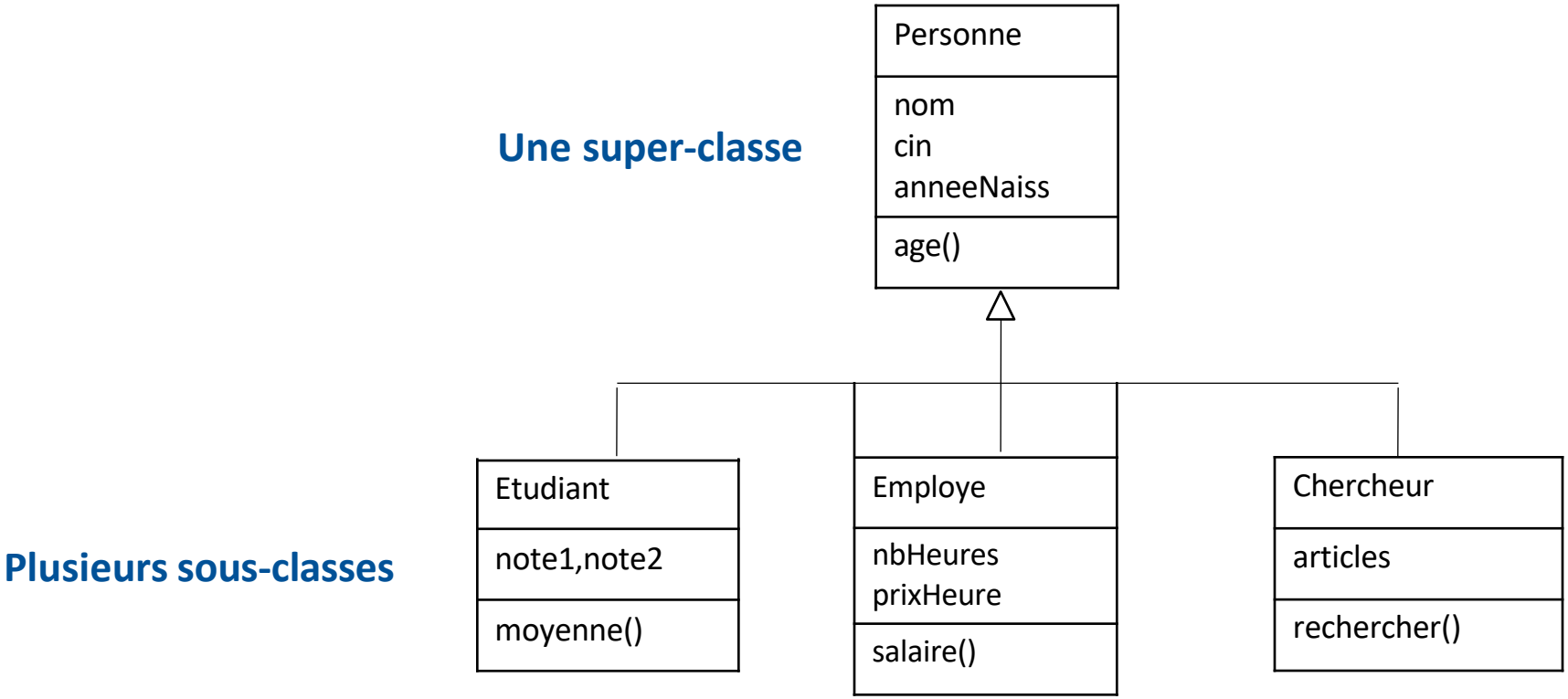
- En Java, on indique qu'une classe hérite d'une autre en utilisant le mot-clé "***extends***"

```
class Personne {

String nom;
int cin,anneeNaiss;
public int age()
{return 2020-anneeNaiss}
}
```

```
class Etudiant extends Personne
{

float note1,note2;

public float moyenne()
{
  return ((note1+note2)/2);
}

}
```

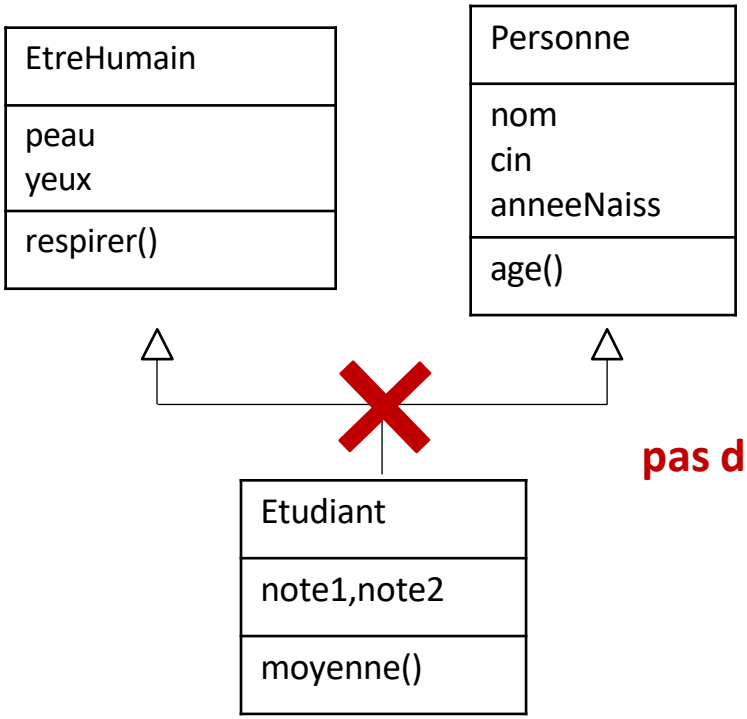
L'héritage en java

- Une classe peut avoir <u>plusieurs</u> sous-classes.



L'héritage en java

- Une classe ne peut avoir qu'<u>une seule classe mère</u>

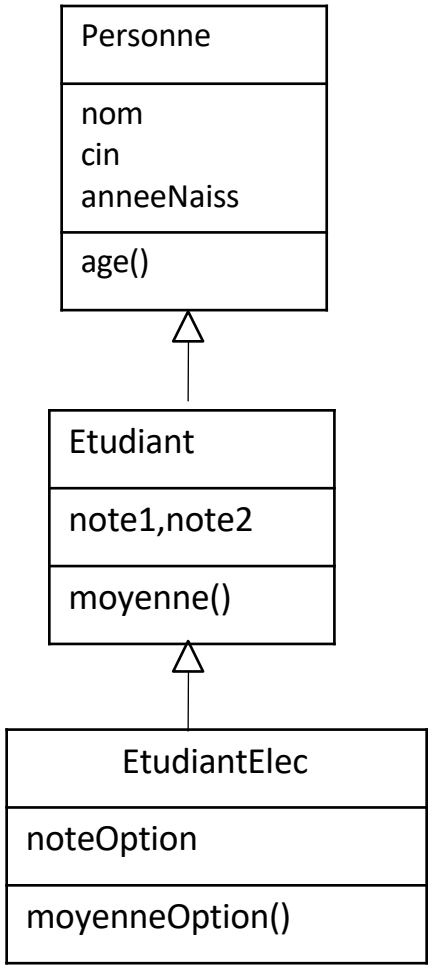


pas d'héritage multiple en java

```
class Etudiant extends Personne, EtreHumain
```

L'héritage en java

Héritage en cascade



Etudiant hérite de *Personne*
EtudiantElec hérite de *Etudiant*

} *EtudiantElec* hérite de *Personne*

```
Etudiant etudiant=new Etudiant()
etudiant.age();
etudiant.moyenne()
etudiant.moyenneOption();
```

```
EtudiantElec etudiantElec=new EtudiantElec()
etudiantElec.age();
etudiantElec.moyenne()
etudiantElec.moyenneOption();
```

Héritage et encapsulation

Package 1

```
public class Personne {

public String nom;
protected String prenom;
int  age;
private long cin;
}
```

```
class Etudiant extends Personne{
…
void f(){

}
…}
```

Package 2

```
public class Employe extends Personne{
…
  void g()
  {
  }
…}
```

On ne peut pas accéder à ***cin*** ni à ***age***

On ne peut pas accéder à ***cin***

Le mot-clé super

- Le mot-clé *super* permet de désigner la superclasse
- *super* permet d'accéder aux attributs et méthodes de la super-classe

- Pour manipuler un attribut de la super-classe :

```
super.cin
```

- Pour manipuler une méthode de la super-classe :

```
super.lire()
```

- Pour faire appel au constructeur de la super-classe:

```
super(nom, cin)
```

Constructeurs et héritage

Constructeur sans argument

```
class Personne {

private String nom;
private long cin;

public Presonne(){
        nom="Mohamed";
        cin=00000000;
}

}
```

```
class Etudiant extends Personne
{
    private float note1,note2;

     public Etudiant(){
         super();
         note1=10.0f;
         note2=10.0f;}


     public Etudiant(float
note1,float note2){
         super();
         this.note1=note1;
         this,note2=note2;}


}
```

super fait appel au constructeur sans argument de la classe mère

Constructeurs et héritage

Constructeur avec arguments

```
class Personne {

private String nom;
private long cin;

public Presonne(String nom,
long cin){
        this.nom=nom;
        this.cin=cin;
}

}
```

```
class Etudiant extends Personne {

  private float note1,note2;

public Etudiant(String nom,long   cin){
        super(nom,cin);
        note1=10.0f;
        note2=10.0f;}
}
public Etudiant(String nom,long cin,
float note1,float note2){
        super(nom,cin);
        this.note1=note1;
        this.note2= note2;}
}
```

- super fait appel au constructeur avec arguments de la classe mère
- La **première** instruction dans le constructeur de la sous-classe doit être l'appel au constructeur de la superclasse avec le mot clé **super**.

Si on ne fait pas d'appel explicite au constructeur de la superclasse, c'est le constructeur par défaut (ou sans argument) de la superclasse qui est appelé implicitement.

Constructeurs et héritage

```
public class Animal {

   public int nbPattes;

   public Animal(int nbPattes) {
       this.nbPattes = nbPattes; }
   }
```

```
public class Chat extends Animal{

}
```

Question:

Testez l'exemple suivant.

Qu'est ce que vous aurez en résultat?

Constructeurs et héritage

```
public class Animal {

   public int nbPattes;

   public Animal(int nbPattes) {
       this.nbPattes = nbPattes;
}
   }
```

```
public class Chat extends Animal{

    public Chat(){
       super();
    }
 }
```

Constructeur par défaut

Appel implicite à super

```
Implicit super constructor
Animal() is undefined for
default constructor. Must
define an explicit
constructor
```

- Dans notre cas, comme nous avons écrit un constructeur avec arguments pour la classe *Animal*, le constructeur *par défaut (ou sans argument)* n'existe pas.
- On a alors une erreur à la compilation lors de l'exécution de *super()*

Constructeurs et héritage

Solution 1: Déclarez explicitement le constructeur par défaut de la classe mère

```
public class Animal {

    public int nbPattes;

    public Animal(){}

    public Animal(int nbPattes) {
        this.nbPattes = nbPattes; }
    }
```

```
public class Chat extends Animal{

}
```

Constructeurs et héritage

Solution 2: Faire un appel explicite au constructeur surchargé de la classe mère

```
public class Animal {

   public int nbPattes;

    public Animal(int nbPattes) {
        this.nbPattes = nbPattes;
     }
 }
```

```
public class Chat extends Animal{

   public Chat(){
       super(4);
   }
 }
```

Constructeurs et héritage

Enchaînement des constructeurs:

```
public class Humain{
    public Humain() {
        Sytem.out.println("Je suis un être humain");
     }
 }
```

```
public class Client extends Humain {
    public Client (){
         Sytem.out.println("Je suis un client");
    }
 }
```

```
public class ClientFidele extends Client {
    public ClientFidele(){
         Sytem.out.println("Je suis un client fidèle");
    }
 }
```

Constructeurs et héritage

Enchaînement des constructeurs:

```
ClientFidele cFidele=new ClientFidele();
```

```
Je suis un être humain
Je suis un client
Je suis un client fidèle
```

Constructeurs et héritage

Enchaînement des constructeurs:

```
public class Humain{
    public Humain() {
        super() ;
        Sytem.out.println("Je suis un être humain");
    }
}
```

```
public class Client extends Humain {
    public Client (){
        super();
        Sytem.out.println("Je suis un client");
    }
}
```

Appel implicite à super

```
public class ClientFidele extends Client {
    public Client Fidele(){
        super() ;
        Sytem.out.println("Je suis un être client fidèle");
    }
}
```

Redéfinition des méthodes

```
class Personne {

      public void lire() {
        System.out.println("Je sais lire");
      }
}
```

```
class Etudiant extends Personne {

 public void lire() {
   System.out.println("Je suis un étudiant");
   System.out.println("Je sais lire");
     }
}
```

- La classe Etudiant hérite la méthode lire() mais on veut changer son traitement
- On garde **la même signature** de la méthode lire et on change le corps

On parle alors de **redéfinition (override)**

Redéfinition des méthodes

```
class Personne {

     public void lire() {
       System.out.println("Je sais lire");
     }
}
```

```
class Etudiant extends Personne {

 public void lire() {
       System.out.println("Je suis un étudiant");
       super.lire();
     }
}
```

Possibilité de réutiliser le code de la méthode héritée (super)

Surcharge vs redéfinition

```
class Personne {

    public void lire() {
            System.out.println("Je sais lire");
    }
}
```

surcharge

```
class Etudiant extends Personne {

 public void lire(String titreLivre) {
            System.out.println("Je sais lire");
            System.out.println("J'ai lu le livre "+titreLivre);
    }
}
```

Etudiant possède deux méthodes

- `lire()`
- `lire(String)`

redéfinition

```
class Etudiant extends Personne {

 public void lire() {
            System.out.println("Je suis un étudiant");
            super.lire();
    }
}
```

Etudiant possède une seule méthode

- `lire()`

Le mot clé final

Empêcher la rédifinition:

- Une classe peut protéger une méthode afin d'éviter qu'elle ne soit redéfinie dans ses sous-classes.
- En Java, on va simplement ajouter le modificateur **final** dans la signature de la méthode.

```
public final void lire()
```

Empêcher l'héritage:

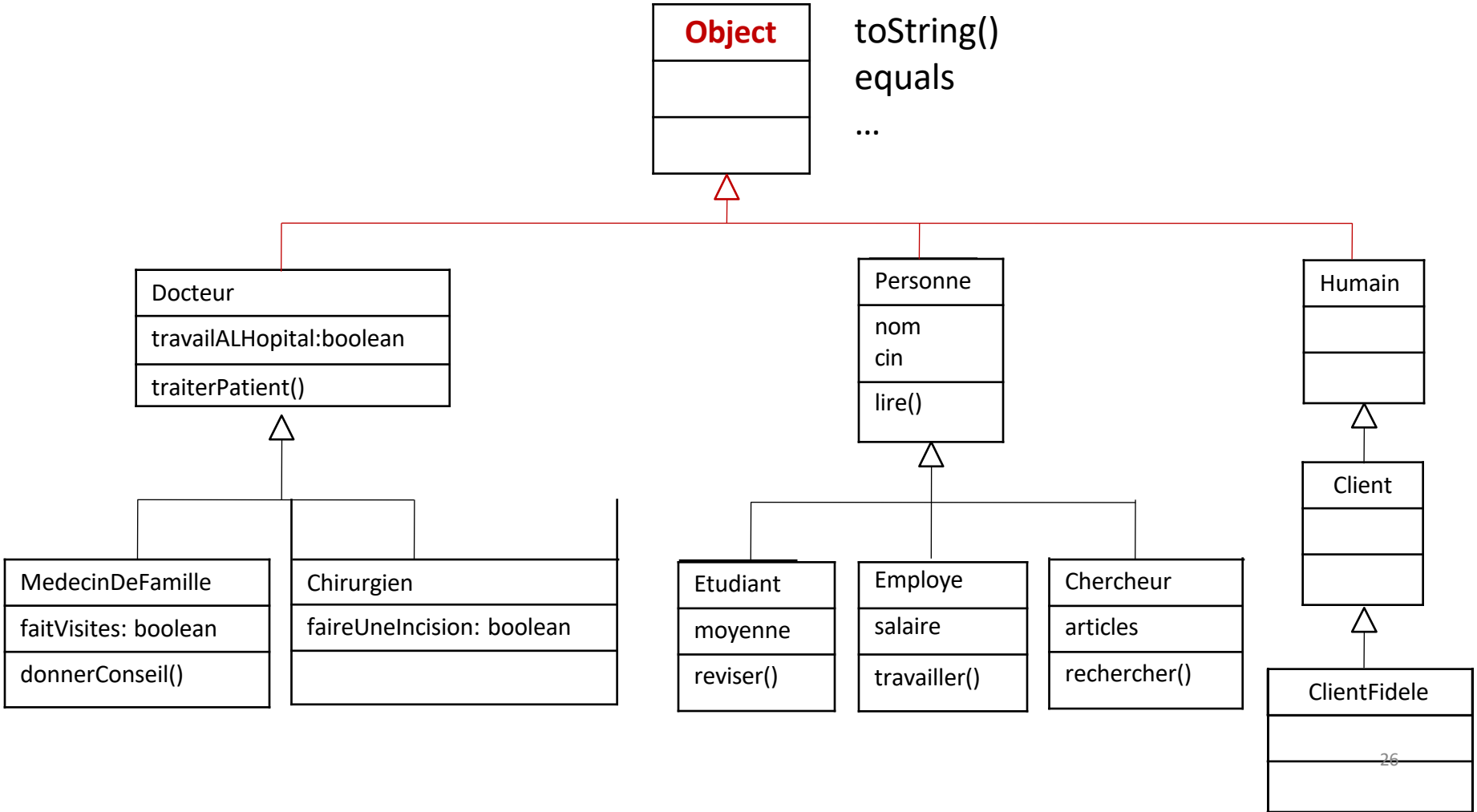
- Une classe déclarée final ne peut pas être dérivée

```
class final Personne
```

  - Permet d'interdire tout héritage pour cette classe qui ne pourra pas être une classe mère
  - Toutes les méthodes à l'intérieur de la classe final seront implicitement finales

La classe Object

- Toutes les classes java héritent implicitement de la classe ***Object*.**
- On dit alors que la classe *Object est l'ancêtre* de toutes les classes



La classe Object

- Méthodes importantes de la classe Object :

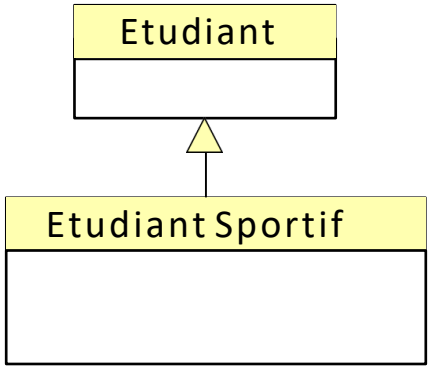
```
public String toString()
public boolean equals ( Object o )
public final Class getClass()
public int hashcode()
```

Partie 2
Polymorphisme

Surclassement

- La réutilisation du code est un aspect important de l'héritage, mais ce n'est peut être pas le plus important
- Le deuxième point **fondamental** est la relation qui relie une classe à sa super-classe :

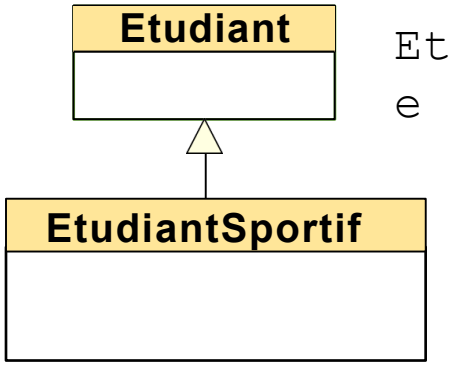
Une classe B qui hérite de la classe A peut être vue comme un sous-type (sous ensemble) du type défini par la classe A.



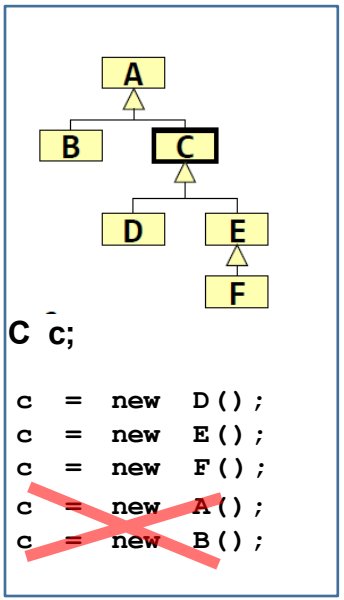
Un `EtudiantSportif` *est un* `Etudiant`

Surclassement

- tout objet instance de la classe B peut être aussi vu comme une instance de la classe A.
- Cette relation est directement supportée par le langage JAVA :
  - *à une référence déclarée de type A, il est possible d'affecter une valeur qui est une référence vers un objet de type B (**surclassement ou upcasting**)*




```
Etudiant e;
e = new EtudiantSportif(...);
```



- ***plus généralement** à une référence d'un type donné, il est possible d'affecter une valeur qui correspond à une référence vers un objet dont le type effectif est n'importe quelle sous-classe directe ou indirecte du type de la référence*

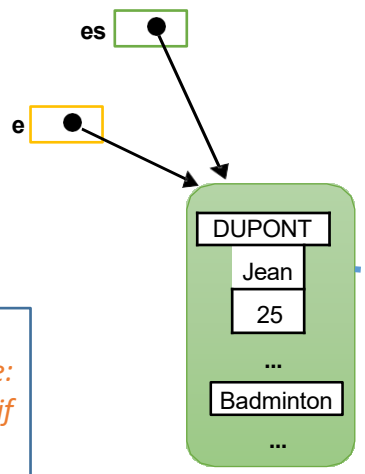
Surclassement

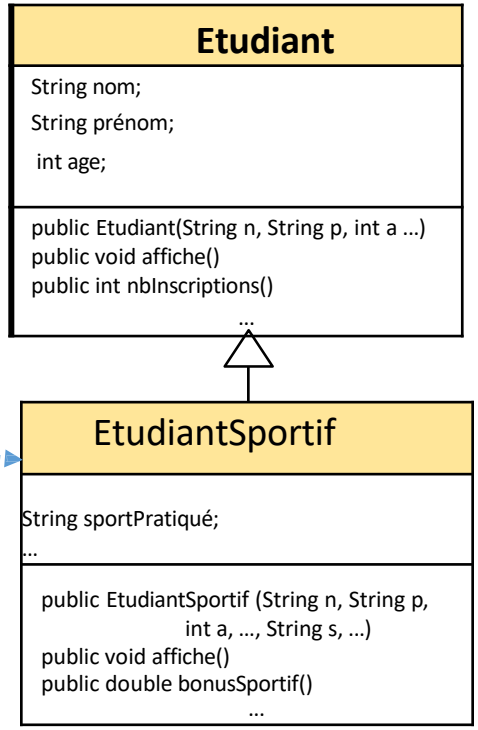
- Lorsqu'un objet est "sur-classé" il est vu par le compilateur comme un objet du type de la référence utilisée pour le désigner
  - *Ses fonctionnalités sont alors restreintes à celles proposées par la classe du type de la référence*

```
EtudiantSportif  es;
es=new     EtudiantSportif("DUPONT","Jean",25,..,"Badminton",..);
Etudiant  e;
e  =  es;// upcasting
e.affiche();
es.affiche();
e.nbInscriptions();
es.nbInscriptions();
es.bonusSportif();
```

~~e.bonusSportif();~~

Le compilateur refuse ce message: pas de méthode bonusSportif définie dans la classe Etudiant





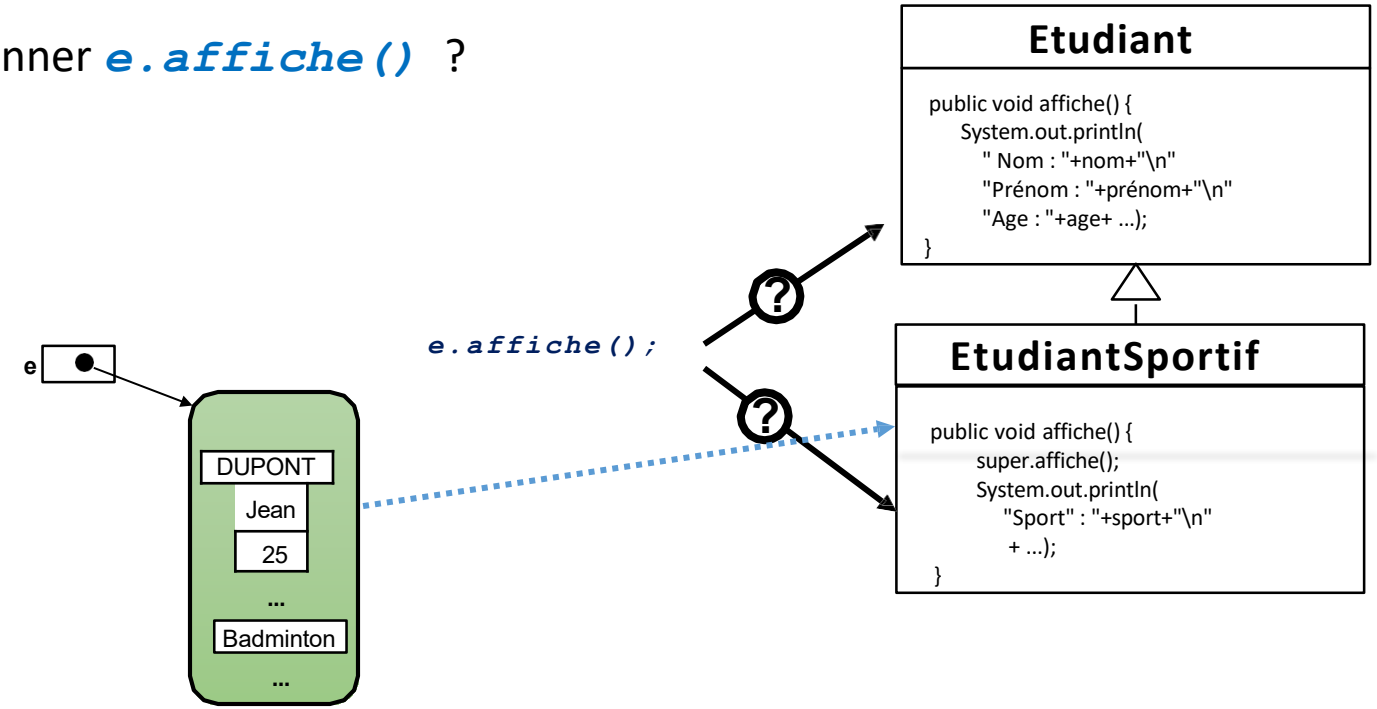
Lien dynamique

```
Etudiant e=new EtudiantSportif("DUPONT","Jean",25,..,"Badminton",..);
```

Type déclaré

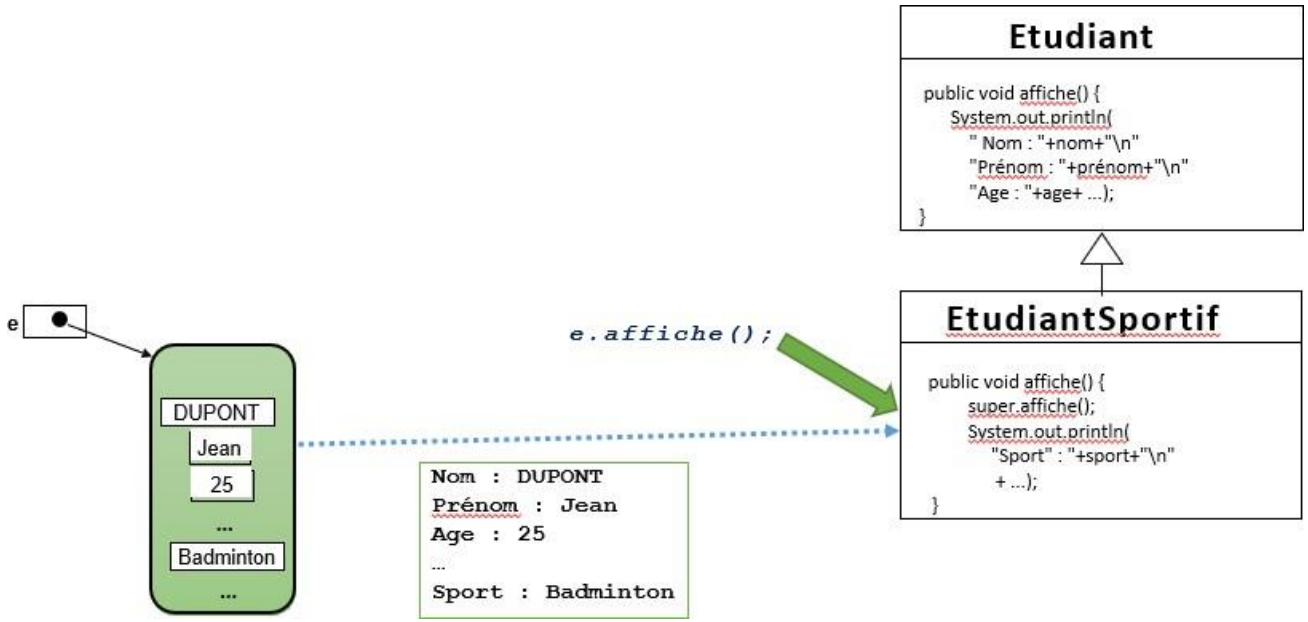
Type effectif

Que va donner ***e.affiche()*** ?



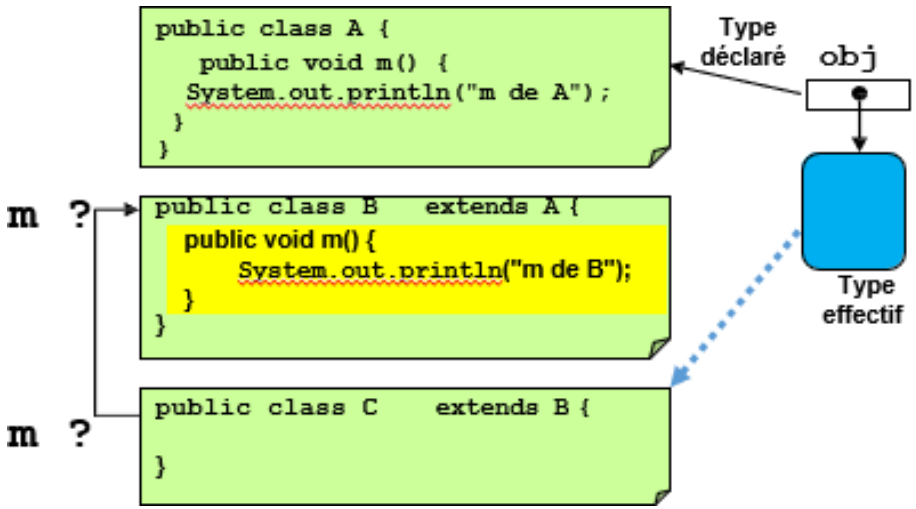
Lien dynamique

- Lorsqu'une méthode d'un objet est accédée au travers d'une référence "surclassée", c'est la méthode telle qu'elle est définie au niveau de **la classe effective** de l'objet qui est en fait invoquée et exécutée



- La méthode exécutée est déterminée à **l'exécution** (run-time) et non pas à la compilation
- ce mécanisme est désigné sous le terme de **lien-dynamique** (*dynamic binding)*

Lien dynamique




```
A obj = new C();
obj.m();
```

m de B

Lien dynamique

- **A la compilation**: seules **des vérifications statiques** qui se basent sur le type déclaré de l'objet (de la référence) sont effectuées.
- La classe déclarée de l'objet doit posséder une méthode dont la signature correspond à la méthode appelée.

```
public class A {
public void m1() {
   System.out.println("m1 de A");
}
}
```

```
public class B    extends A {
public void m1() {
    System.out.println("m1  de B");
}
public void m2() {
    System.out.println("m2  de B");
}
}
```

Type déclaré **obj**

```
A obj = new B();
obj.m1();
obj.m2();
```

- **Le choix de la méthode** à exécuter est effectué **statiquement à la compilation** en fonction du type des paramètres
- **La sélection du code** à exécuter est effectuée **dynamiquement à l'exécution** en fonction du type effectif

Polymorphisme

Q: A quoi servent l'upcasting et le lien dynamique ?

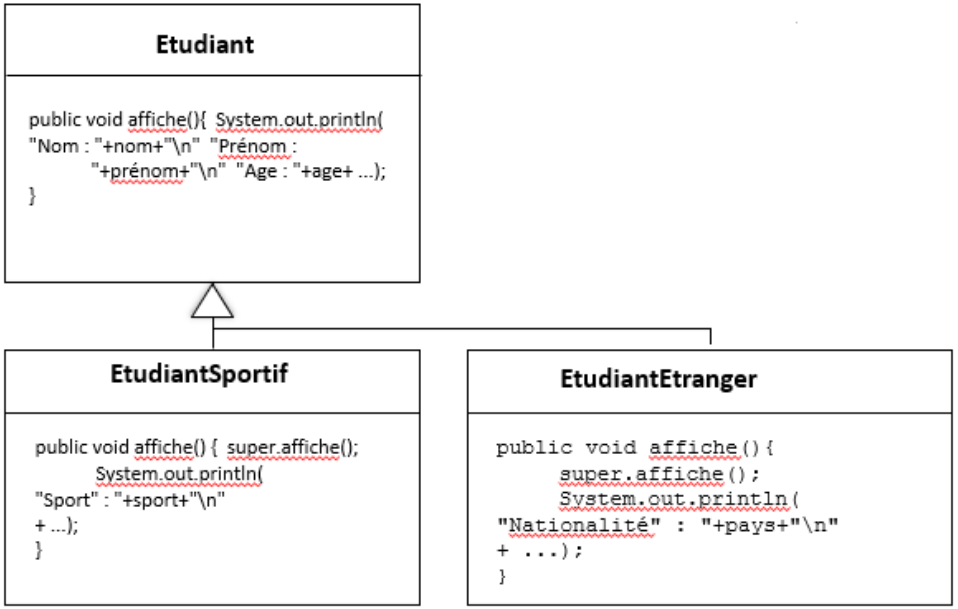
R: A la mise en œuvre du polymorphisme

- Le terme polymorphisme décrit la caractéristique d'un élément qui peut se présenter sous différentes formes.

- En programmation par objets, on appelle polymorphisme:
  - *le fait qu'un objet d'une classe puisse être manipulé comme s'il appartenait à une autre classe*
  - *le fait que la même opération puisse se comporter différemment sur différentes classes de la hiérarchie.*

- Le **polymorphisme** constitue la ***troisième caractéristique*** essentielle d'un langage orienté objet après l'**encapsulation** et l'**héritage**.

Polymorphisme




```
public class GroupeTD{
  Etudiant[] liste = new Etudiant[30];
  int nbEtudiants = 0;
  ...
  public void ajouter(Etudiant e) {
   if (nbEtudiants < liste.lenght)
            liste[nbEtudiants++] = e;
  }

 public  void afficherListe(){
 for (int i=0;i<nbEtudiants; i++)
     liste[i].affiche();
 }

}
```

```
GroupeTD td1 = new GroupeTD();
td1.ajouter(new Etudiant("DUPONT", …));
td1.ajouter(new EtudiantSportif("BIDULE","Louis", … , "ski alpin");
```

- *`liste` peut contenir des étudiants de n'importe quel type*
- *Si un nouveau type d'étudiant est défini,le code de `GroupeTD` reste inchangé*

Polymorphisme

- Développement plus rapide
- Plus grande simplicité et meilleure organisation du code
- Programmes plus facilement **extensibles**
- **Maintenance** du code plus aisée

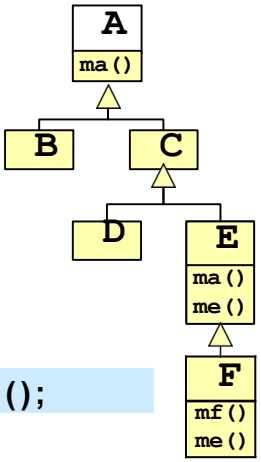
Downcasting

```
ClasseX   obj    =      …
ClasseA   a=(ClasseA)obj;
```

- Le downcasting (ou transtypage) permet de « forcer un type » **à la compilation**
  - C'est une « ***promesse*** » que l'on fait au moment de la compilation.
- Pour que le transtypage soit valide, *il faut qu'à l'exécution le type effectif de obj soit « compatible » avec le type ClasseA*
  - Compatible : la même classe ou n'importe quelle sous classe de *ClasseA* (obj ***instanceof*** ClasseA)
- Si la promesse n'est pas tenue une erreur d'exécution se produit.
  - ***ClassCastException*** est levée et arrêt de l'exécution

On peut s'assurer qu'un objet est bien une instance d'une classe donnée en recourant à l'opérateur ***instanceOf***.

Upcasting/Downcastin




```
class A {
    public void ma() {
        System.out.println("methode ma définie dans A");
    }
}
```

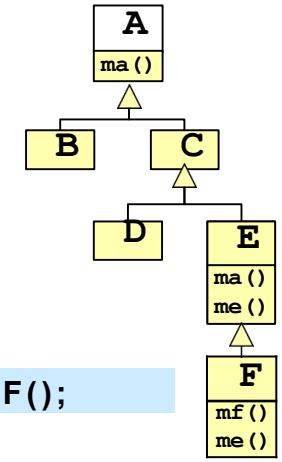
```
class E extends C {
    public void ma() {
        System.out.println("methode ma redéfinie dans E");
    }
    public void me() {
        System.out.println("methode me définie dans E");
    }
}
```

```
class F extends E {
    public void mf() {
        System.out.println("methode mf définie dans f");
    }
    public void me() {
        System.out.println("methode me redéfinie dans F");
    }
}
```

C c = new F ( ) ;

	compilation	exécution
c.ma();		
c.mf ( ) ;		
Bb = c;		
Ee = c;		
Ee = (E) c; e.me();		
Dd = (D) c;		

Upcasting/Downcastin




```
class A {
    public void ma() {
        System.out.println("methode ma définie dans A");
    }
}
```

```
class E extends C {
    public void ma() {
        System.out.println("methode ma redéfinie dans E");
    }
    public void me() {
        System.out.println("methode me définie dans E");
    }
}
```

```
class F extends E {
    public void mf() {
        System.out.println("methode mf définie dans f");
    }
    public void me() {
        System.out.println("methode me redéfinie dans F");
    }
}
```

C c = new F ( ) ;

	compilation	exécution
c.ma();	La classe C hérite d'une méthode ma	→ méthode ma définie dans E
c.mf ( ) ;	*Cannot find symbol : metod mf()* Pas de méthode mf() définie au niveau de la classe C	
Bb = c;	*Incompatible types* Un C n'est pas un B	
Ee = c;	*Incompatible types* Un C n'est pas forcément un E	
Ee = (E) c; e.me();	Transtypage (Dowcasting), le compilateur ne fait pas de vérification La classe E définit bien une méthode me	→ méthode me définie dans F
Dd = (D) c;	Transtypage (Dowcasting), le compilateur ne fait pas de vérification	*ClassCastException* Un F n'est pas un D

Partie 3
classe abstraite

Classes abstraites

- Une classe abstraite est une classe qui ne permet pas d'instancier des objets. Elle ne peut servir que de classe de base pour une dérivation.
- On ne connaît pas toujours le comportement par défaut d'une opération commune à plusieurs sous-classes
- Méthode abstraite et conséquences :
  - Si une seule des méthodes d'une classe est abstraite, alors la classe devient aussi abstraite
  - Toutes les classes filles héritant de la classe mère abstraite doivent implémenter toutes ses méthodes abstraites ou sinon elles sont aussi abstraites

Classes abstraites et Java

- Le mot clé **abstract** est utilisé pour spécifier qu'une classe est abstraite
- Une classe abstraite se déclare ainsi:

```
public abstract class NomMaClasse {... }
```

- Une méthode abstraite se déclare ainsi

```
public abstract void maMethode(...);
```

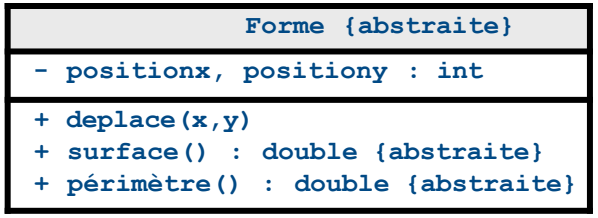
Pour créer une méthode abstraite, la signature (nom et paramètres) est déclarée sans spécifier le corps et en ajoutant le mot clé abstract

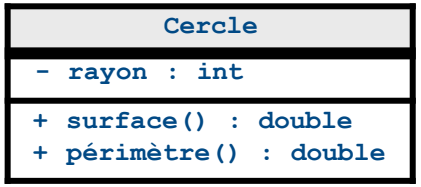
Classes abstraites : exemple Forme

Exemple : la classe *Forme*

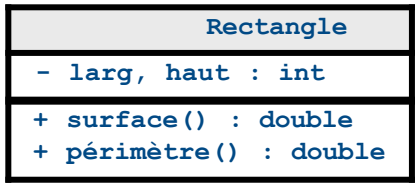
✓ Les méthodes *surface()* et *périmètre()* sont abstraites

✓ Ces méthodes n'ont de « sens » que pour les sous-classes *Cercle* et *Rectangle*






```
public abstract class Forme {  private int
    positionx, positiony;

public void deplacer(double dx, double dy){   x
    += dx; y += dy;
}

    public abstract double périmètre();
    public abstract double surface();
}
```



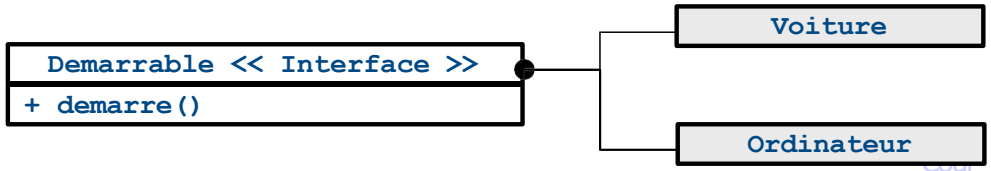
Pas d'implémentation !!

Partie 4
interface

Notion d'interface

- Une interface est un modèle pour une classe
- Quand toutes les méthodes d'une classe sont abstraites et qu'il n'y a aucun attribut nous aboutissons à la notion d'interface
- Elle définit la signature des méthodes qui doivent être implémentées dans les classes qui respectent ce modèle
- Toute classe qui implémente l'interface doit implémenter toutes les méthodes définies par l'interface
- Tout objet instance d'une classe qui implémente l'interface peut être déclaré comme étant du type de cette interface
- Les interfaces pourront se dériver

Exemple: Les choses *Demarrable* doivent posséder une méthode *demarre()*



Notion d'interface et java

- Mise en œuvre d'une interface
  - La définition d'une interface se présente comme celle d'une classe. Le mot clé **interface** est utilisé à la place de **class**

```
public interface NomInterface {…}
```

Interface : ne pas confondre avec les interfaces graphiques

- Lorsqu'on définit une classe, on peut préciser qu'elle implémente une ou plusieurs interface(s) donnée(s) en utilisant une fois le mot clé implements

```
public class NomClasse implements Interface1,Interface3, ... {
  …
}
}
```

- Si une classe hérite d'une autre classe elle peut également implémenter une ou plusieurs interfaces

```
public class NomClasse extends SuperClasse implements Inter1, ... {…}
```

Notion d'interface et java

- une interface ne comporte que des constantes et des méthodes abstraites.

```java
public interface NomInterface {
  public static final int CONST=2;
  public abstract void f();
}
```

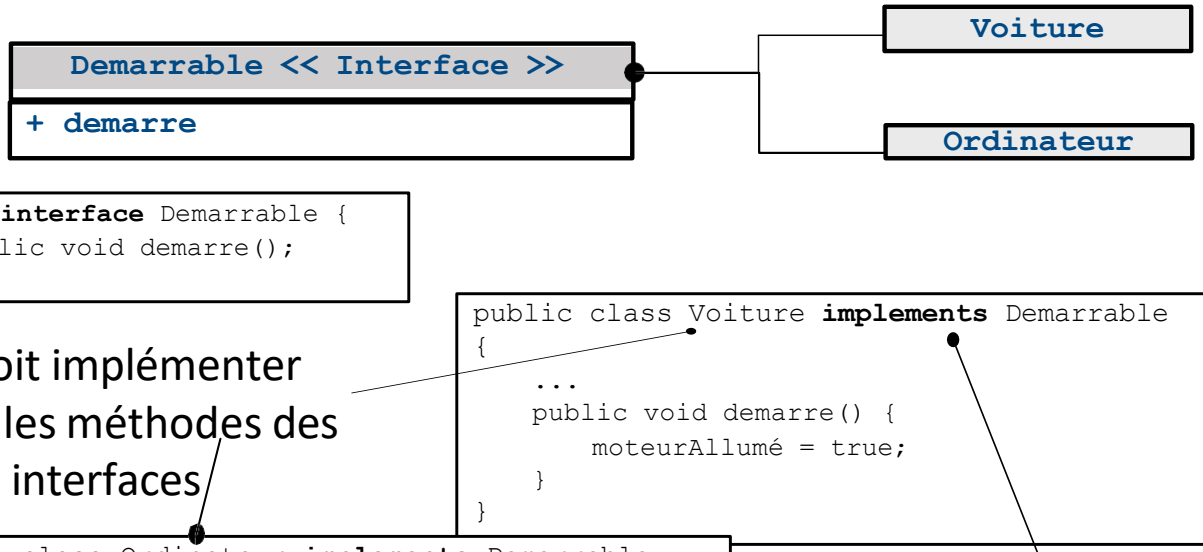
```java
public interface NomInterface {
  int CONST=2;
  void f();
}
```

- Les interfaces ne sont pas instanciables (Même raisonnement avec les classes abstraites)

NomInterface jeTente = new NomInterface(); // Erreur!!

Notion d'interface et java

- Toute classe qui implémente l'interface doit implémenter <u>toutes</u> les méthodes définies par l'interface




```
public interface Demarrable {
    public void demarre();
}
```

```
public class Voiture implements Demarrable
{
    ...
    public void demarre() {
        moteurAllumé = true;
    }
}
```

Doit implémenter toutes les méthodes des interfaces

```
public class Ordinateur implements Demarrable
{
    ...
    public void demarre() {
        estDemarree = true;
    }
}
```

Une « Voiture » est « Demarrable »

Un « Ordinateur » est « Demarrable »

Notion d'interface et java

- Tout objet instance d'une classe qui implémente l'interface peut être déclaré comme étant du type de cette interface

```
public class Test {
    public static void main (String[] argv) {
        // Déclaration d'un objet de type Demarrable
        Demarrable dem1;
        // Création d'un objet Voiture
        dem1 = new Voiture();

        // Déclaration et création d'un objet Personne
        Personne pers1 = new Personne(dem1);
        pers1.mettreEnRoute();

        // Déclaration d'un objet de type Demarrable
        Demarrable dem2;
        // Création d'un objet Ordinateur
        dem2 = new Ordinateur();

        // Déclaration et création d'un objet Personne
        Personne pers2 = new Personne(dem2);
        pers2.mettreEnRoute();
        }
        }
```

Une personne peut demarrer tous les objets *Demarrable*

Notion d'interface et java

Exemple : une *Voiture* et un *Ordinateur* sont des objets *Demarrable*

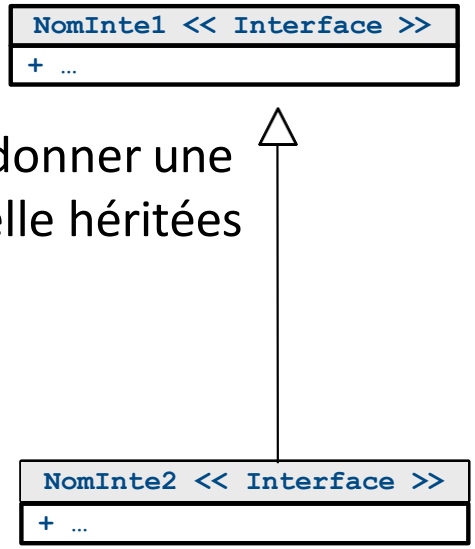
```
public class Person {

private Demarrable objetDemarrable;

public Person(Demarrable dem) {
objetDemarrable = dem;
}

public void mettreEnRoute() {   objetDemarrable.demarre();
}
}
```

Une personne peut démarrer *Voiture* et *Ordinateur* sans connaître leur nature exacte

Notion d'interface et java

- Les interfaces pourront sedériver
  - Une interface peut hériter d'une autre <u>interface</u> : **« extends »**

- Conséquences
  - Toute classe qui implémente l'interface fille doit donner une implémentation à toutes les méthodes mêmes celle héritées



- Utilisation
  - Lorsqu'un modèle peut se définir en plusieurs sous-modèles complémentaires

Classes VS Classes abstraites VS interfaces

- Les classes
  - Elles sont complètement implémentées
  - Une autre classe peut en hériter
- Les classes abstraites
  - Sont partiellement implémentées
  - Une autre classe non abstraite peut en hériter mais doit donner une implémentation aux méthodes abstraites
  - Une autre classe abstraite peut en hériter sans forcément donner une implémentation à toutes les méthodes abstraites
  - Ne peuvent pas être instanciées mais peuvent fournir un constructeur
- Les interfaces
  - Elles ne sont pas implémentées
  - Toute classe qui implémente une ou plusieurs interfaces doit implémenter toutes leurs méthodes (abstraites)

Classes VS Classes abstraites VS interfaces

- Les classes
  - Elles sont complètement implémentées
  - Une autre classe peut en hériter
- Les classes abstraites
  - Sont partiellement implémentées
  - Une autre classe non abstraite peut en hériter mais doit donner une implémentation aux méthodes abstraites
  - Une autre classe abstraite peut en hériter sans forcément donner une implémentation à toutes les méthodes abstraites
  - Ne peuvent pas être instanciées mais peuvent fournir un constructeur
- Les interfaces
  - Elles ne sont pas implémentées
  - Toute classe qui implémente une ou plusieurs interfaces doit implémenter toutes leurs méthodes (abstraites)

Interface java 8

- Java 8 modifie le concept d'interface, en permettant à une interface d'avoir:
  - **Des méthodes par défaut (default)**
  - **Des méthodes statiques (static)**

Interface java 8: Méthode par défaut

- Pour créer une méthode par défaut dans l'interface Java, nous devons utiliser le mot-clé *« default »* avec la signature de la méthode.
- Par exemple,

```
public interface Interface1 {

  void method1(String str);

  default void log(String str){
      System.out.println("I1 logging::"+str);
  }
}
```

- lorsqu'une classe implémentera Interface1, il <u>n'est pas obligatoire</u> de fournir l'implémentation des méthodes par défaut de l'interface.
- Les méthodes par défaut nous aidera à étendre les interfaces avec des méthodes supplémentaires,

Interface java 8- Méthode par défaut

- Si nous avons une autre interface avec les méthodes suivantes :

```
public interface Interface2 {
  void method2();
  default void log(String str){
   System.out.println("I2 logging::"+str);}}
```

- Et si une classe implémente à la fois Interface1 et Interface2 et n'implémente pas la méthode commune par défaut, **le compilateur ne peut pas décider laquelle choisir**.
- Donc, il est obligatoire de fournir une implémentation pour les méthodes par défaut des interfaces.

```
public class MyClass implements Interface1, Interface2 {
  @Override
  public void method2() {      }
  @Override
  public void method1(String str) {  }
  @Override
  public void log(String str){
      System.out.println("MyClass logging::"+str);
     Interface1.log("abc");    }}
```

Interface java 8: Méthode statique

- La méthode statique de l'interface Java est similaire à la méthode par défaut, sauf que nous ne pouvons pas les remplacer dans les classes d'implémentation.
- Cette fonctionnalité nous aide à éviter les résultats indésirables en cas de mauvaise implémentation dans les classes d'implémentation.

Examinons cela avec un exemple simple:

```
public interface MyData {
  default void print(String str) {
   if (!isNull(str))
         System.out.println("MyData Print::" + str);
  }
static boolean isNull(String str) {
  System.out.println("Interface Null Check");
  return str == null ? true : "".equals(str) ? true : false;
  }}
```

Interface java 8: Méthode statique

- Voyons maintenant une classe d'implémentation qui a la méthode isNull() avec une mauvaise implémentation.

```
public class MyDataImpl implements MyData {
  public boolean isNull(String str) {
   System.out.println("Impl Null Check");
   return str == null ? true : false;}
  public static void main(String args[]){
   MyDataImpl obj = new MyDataImpl();
   obj.print("");
   obj.isNull("abc");
  }}
```

- Notez que **isNull(String str)** est une méthode de classe MyDataImpl, elle ne remplace pas la méthode d'interface.
- si nous ajoutons l'annotation @Override à la méthode isNull(), cela entraînera une erreur du compilateur.
- Maintenant, lorsque nous allons exécuter l'application, nous obtenons la sortie suivante:

Interface Null Check
Impl Null Check

Interface java 8: Méthode statique

- Si nous transformons la méthode d'interface de statique en valeur par défaut, nous obtiendrons la sortie suivante:

  Impl Null Check
  MyData Print::
  Impl Null Check

- La méthode statique de l'interface Java n'est visible que pour les méthodes d'interface, si nous supprimons la méthode isNull() de la classe MyDataImpl, nous ne pourrons pas l'utiliser pour MyDataImpl objet.
- Cependant, comme d'autres méthodes statiques, nous pouvons utiliser des méthodes statiques d'interface en utilisant le nom de la classe. Par exemple: `boolean result = MyData.isNull("abc");`

- Les méthodes statique permet d'améliorer la sécurité, car les classes filles ne peuvent pas redéfinir la méthode statique

Interface java 8- interfaces fonctionnelles

- Une interface fonctionnelle en Java est une interface qui ne contient qu'une seule méthode abstraite (non implémentée).
- Une interface fonctionnelle peut contenir des méthodes par défaut et statiques qui ont une implémentation, en plus de la méthode unique non implémentée.
- Voici un exemple d'une interface fonctionnelle en Java:

```
public interface InterfaceFonctionnel {
    public void show();
}
```

- L'interface ci-dessus compte comme une interface fonctionnelle car il ne contient qu'une seule méthode, et cette méthode n'a pas d'implémentation.
- Normalement, une interface Java ne contient pas d'implémentations des méthodes qu'elle déclare, mais elle peut contenir des implémentations dans les méthodes par défaut ou dans les méthodes statiques.
- Voici un autre exemple d'une interface fonctionnelle en Java, avec des implémentations de certaines des méthodes:

```
public interface InterfaceFonctionnel {
   public void show();
   public default int addition(int a, int b) {return a + b;}
   public static int soustraction(int a, int b) {return a - b;}
}
```

Interface java 8- interfaces fonctionnelles

- Les interfaces fonctionnelles peuvent être implémentées par une expression Lambda Une interface fonctionnelle peut être implémentée par une expression Lambda.
- Voici un exemple qui implémente l'interface fonctionnelle « InterfaceFonctionnel » définie dans le code ci-dessus:

```
InterfaceFonctionnel lambda = () -> {
    System.out.println("Calculatrice");
}
```

- Une expression Lambda n'implémente qu'une seule méthode à partir d'une interface Java.
- Afin de savoir quelle méthode l'expression Lambda implémente, l'interface ne peut contenir qu'une seule méthode non implémentée. En d'autres termes, l'interface doit être une interface fonctionnelle.

Merci pour votre attention